Complete GenAI Engineering Ecosystem Map

For Building Enterprise RAG + Agentic AI Systems

Target Role: Gen AI Engineer at Pahlé India Foundation (PIF)

---

1. THE BIG PICTURE

A modern GenAI application is NOT just:

```
User → ChatGPT → Response
```

Enterprise systems are actually:

```
User Query
    ↓
Frontend/UI
    ↓
Backend API Layer
    ↓
Authentication + Validation
    ↓
Agent / Orchestration Layer
    ↓
Retrieval System (RAG)
    ↓
Vector Database + BM25
    ↓
Document Processing Pipeline
    ↓
PDFs / Policy Documents / Databases / APIs
```

The biggest realization you need:

> GenAI Engineering is mostly SYSTEM DESIGN.

NOT model training.

---

2. COMPLETE GENAI ENGINEERING GRAPH

```
                         ┌───────────────────────┐
                         │        USERS          │
                         │ Researchers/Analysts  │
                         └───────────┬───────────┘
                                     │
                                     ▼
                    ┌─────────────────────────────────────┐
                    │         FRONTEND / UI              │
                    │ Streamlit / Next.js / React        │
                    └─────────────────┬───────────────────┘
                                      │
                                      ▼
            ┌───────────────────────────────────────────────┐
            │          BACKEND API LAYER                   │
            │ FastAPI / Flask / Async Endpoints            │
            └─────────────────┬─────────────────────────────┘
                              │
                              ▼
    ┌──────────────────────────────────────────────────────────────┐
    │           ORCHESTRATION / AI LOGIC                          │
    │ LangChain / LlamaIndex / LangGraph / CrewAI                │
    └─────────────────┬──────────────────────────────────────────┘
                      │
         ┌────────────┼────────────────────────┐
         │            │                        │
         ▼            ▼                        ▼
┌────────────────┐ ┌───────────────────┐ ┌──────────────────────┐
│ Prompt Layer   │ │ Agent Layer       │ │ Retrieval Layer      │
│ Templates       │ │ Tool Calling       │ │ RAG Pipelines         │
│ Output Parsing  │ │ Multi-step Logic   │ │ Context Injection     │
└────────┬───────┘ └─────────┬─────────┘ └──────────┬───────────┘
         │                   │                       │
         ▼                   ▼                       ▼
┌────────────────┐ ┌───────────────────┐ ┌──────────────────────┐
│ LLM APIs       │ │ External Tools    │ │ Retriever System     │
│ GPT / Claude   │ │ Search/API/SQL    │ │ Hybrid Search        │
│ Gemini          │ │ Web/Data Tools     │ │ Semantic Retrieval    │
└────────┬───────┘ └─────────┬─────────┘ └──────────┬───────────┘
         │                   │                       │
         ▼                   ▼                       ▼
 ┌──────────────────────────────────────────────────────────────┐
 │                  VECTOR DATABASES                           │
 │ Chroma / FAISS / Pinecone / Weaviate                        │
```

```
└─────────────────────┬───────────────────────────────────────┘
                      │
                      ▼
       ┌──────────────────────────────────────────┐
       │         EMBEDDING MODELS                 │
       │ OpenAI Embeddings / BGE / Gemini         │
       └──────────────────┬───────────────────────┘
                          │
                          ▼
         ┌──────────────────────────────────────┐
         │        DOCUMENT CHUNKING             │
         │ Semantic / Parent-Child / Recursive  │
         └─────────────────┬────────────────────┘
                           │
                           ▼
     ┌──────────────────────────────────────────────────┐
     │           DOCUMENT INGESTION PIPELINE            │
     │ Parsing / OCR / Metadata Extraction              │
     └─────────────────────┬────────────────────────────┘
                           │
                           ▼
    ┌───────────────────────────────────────────────────┐
    │ SOURCE DOCUMENTS                                  │
    │ PDFs / Reports / Whitepapers / Websites / Databases │
    └───────────────────────────────────────────────────┘
```

---

3. FRONTEND LAYER

What It Does

The frontend is the user-facing interface.

This is where:

- users upload PDFs,
- ask questions,
- see citations,
- interact with the AI system.

Technologies

Technology	Purpose
Streamlit	Fast AI app prototyping
Gradio	Simple AI demos
React	Production frontend
Next.js	Enterprise frontend

For Your Projects

Project 1

Use:

- Streamlit

Project 2

Use:

- Streamlit OR Next.js

---

4. BACKEND API LAYER

What It Does

Acts as the central brain of the application.

Handles:

- requests,
- authentication,
- API routing,
- orchestration,
- business logic.

Technologies

Technology	Purpose
FastAPI	High-performance Python backend
Flask	Lightweight backend
Uvicorn	ASGI server

Why FastAPI Matters

FastAPI is the industry standard for GenAI backends because:

- async support,
- fast inference serving,
- clean APIs,
- automatic documentation.

---

5. ORCHESTRATION FRAMEWORKS

This is one of the MOST IMPORTANT layers.

These frameworks coordinate:

- prompts,
- retrieval,
- LLM calls,
- tools,
- agents.

LANGCHAIN

What It Does

Provides abstractions for:

- chains,
- retrievers,
- prompts,
- tools,

- memory,
- agents.

Core Concepts

Concept	Purpose
PromptTemplate	Dynamic prompts
Chains	Sequential workflows
LCEL	Composable pipelines
Retrievers	Search interfaces
Agents	Tool-using reasoning systems

LLAMAINDEX

What It Does

Specializes in:

- document indexing,
- retrieval,
- query engines.

Excellent for:

- large document collections,
- hierarchical retrieval,
- policy PDFs.

LANGGRAPH

What It Does

Advanced agent orchestration.

Allows:

- loops,

- branching,
- stateful workflows,
- memory persistence.

Why Important

Enterprise AI systems are NOT linear.

They require:

```
Reason
→ Search
→ Evaluate
→ Retry
→ Summarize
```

LangGraph manages this.

CREWAI

What It Does

Multi-agent collaboration framework.

Example:

```
Research Agent
        ↓
Summarization Agent
        ↓
Citation Agent
```

---

6. PROMPT ENGINEERING LAYER

What It Does

Controls:

- AI behavior,
- structure,
- output quality,
- hallucinations.

Important Concepts

Concept	Purpose
Few-shot prompting	Provide examples
Chain-of-thought	Step-by-step reasoning
System prompts	Define AI behavior
Structured outputs	JSON/schema responses
Guardrails	Reduce hallucinations

---

7. LLM APIs

What They Do

Generate:

- answers,
- summaries,
- reasoning,
- analysis.

Major Providers

Provider	Strength
OpenAI	Best ecosystem

Provider	Strength
Claude	Long context reasoning
Gemini	Multimodal + Google ecosystem

Important Concepts

Concept	Meaning
Tokens	Units of text
Context window	Memory size
Temperature	Creativity level
Max tokens	Output size limit

---

8. RAG (RETRIEVAL AUGMENTED GENERATION)

MOST IMPORTANT CONCEPT FOR YOUR INTERVIEW

What Problem RAG Solves

LLMs do NOT know your documents.

RAG gives the LLM relevant context dynamically.

RAG FLOW

```
User Query
     ↓
Retriever searches documents
     ↓
Relevant chunks selected
     ↓
Chunks inserted into prompt
```

```
     ↓
LLM generates grounded answer
```

Why RAG Matters

Without RAG:

- hallucinations increase,
- outdated information,
- no document grounding.

With RAG:

- grounded responses,
- citations,
- enterprise reliability.

---

9. DOCUMENT INGESTION PIPELINE

What It Does

Converts raw PDFs into AI-searchable knowledge.

Pipeline

```
PDF
→ Parse text
→ OCR if needed
→ Extract metadata
→ Clean text
→ Chunk documents
→ Generate embeddings
→ Store vectors
```

Technologies

Technology	Purpose
PyMuPDF	PDF parsing
unstructured	Layout-aware parsing
Tesseract	OCR
Camelot	Table extraction

---

10. CHUNKING

What It Does

Splits documents into smaller pieces.

Why Important

Chunking determines:

- retrieval quality,
- hallucination rate,
- context quality.

Chunking Types

Type	Purpose
Recursive chunking	Basic splitting
Semantic chunking	Meaning-aware splitting
Parent-child chunking	Small retrieval + large context
Hierarchical chunking	Preserve document structure

---

11. EMBEDDINGS

What They Do

Convert text into vectors.

Why Important

Allows semantic similarity search.

Example:

```
"AI policy"
≈
"Artificial intelligence regulation"
```

Technologies

Technology	Purpose
OpenAI Embeddings	High-quality embeddings
BGE models	Open-source embeddings
Gemini embeddings	Google embeddings

---

12. VECTOR DATABASES

What They Do

Store embeddings for fast retrieval.

Technologies

Database	Best Use
Chroma	Local development
FAISS	Fast local retrieval
Pinecone	Managed cloud vector DB
Weaviate	Enterprise vector search

Core Operations

Operation	Purpose
Insert vectors	Store knowledge
Similarity search	Retrieve related chunks
Metadata filtering	Filter by source/year/domain

---

13. HYBRID SEARCH

ENTERPRISE-LEVEL RETRIEVAL

Most beginner systems use ONLY vector search.

Enterprise systems combine:

```
BM25 + Dense Retrieval + Reranking
```

BM25

What It Does

Traditional keyword search.

Excellent for:

- exact terms,
- policy names,
- ministries,
- years.

Dense Retrieval

What It Does

Semantic similarity search.

Excellent for:

- meaning,
- paraphrases,
- conceptual similarity.

Reciprocal Rank Fusion (RRF)

What It Does

Combines:

- BM25 rankings,
- vector rankings.

---

14. RERANKING

What It Does

Improves retrieval quality.

Why Important

Initial retrieval is noisy.

Rerankers reorder results using deeper semantic analysis.

Technologies

Technology	Purpose
FlashRank	Fast reranking
Cohere Rerank	High-quality reranking
Cross Encoders	Deep semantic reranking

---

15. AGENTS

What They Do

Allow AI systems to:

- reason,
- use tools,
- perform multi-step workflows.

Example Agent Flow

```
User asks question
    ↓
Agent decides to search documents
    ↓
Agent retrieves context
    ↓
Agent summarizes findings
    ↓
Agent generates final report
```

Tool Examples

Tool	Purpose
Web search	External information
SQL tool	Database querying
Calculator	Computation
Retriever tool	Document search

---

16. MEMORY SYSTEMS

What They Do

Allow agents to remember information.

Types

Memory Type	Purpose
Short-term memory	Conversation context
Long-term memory	Persistent storage
Semantic memory	Meaning-based recall

---

17. EVALUATION FRAMEWORKS

MOST OVERLOOKED AREA IN BEGINNER PROJECTS

Enterprise systems require measurable quality.

RAGAS

What It Measures

Metric	Meaning
Faithfulness	Is answer grounded?
Answer Relevance	Did answer solve query?
Context Precision	Was retrieval relevant?
Context Recall	Did retrieval miss information?

TRULENS

What It Does

Observability + monitoring.

Tracks:

- hallucinations,
- retrieval quality,
- latency.

---

18. OBSERVABILITY & DEBUGGING

What It Does

Allows debugging of:

- prompts,
- retrieval,
- chains,
- agents.

Technologies

Technology	Purpose
LangSmith	Chain tracing
Logging	Error analysis
Metrics	Performance tracking

---

19. FASTAPI ARCHITECTURE

What It Does

Turns AI pipelines into APIs.

Common Architecture

```
routes/
services/
retrieval/
ingestion/
models/
utils/
```

Important Concepts

Concept	Purpose
Routers	Endpoint organization
Pydantic	Request validation
Async endpoints	High performance
Dependency injection	Scalable architecture

---

20. DOCKER & DEPLOYMENT

What It Does

Packages applications into portable containers.

Why Important

Ensures:

- reproducibility,
- deployment consistency,
- scalability.

Deployment Targets

Platform	Purpose
HuggingFace Spaces	Easy demos
AWS	Enterprise cloud
Railway	Quick deployment
Render	Simple hosting

---

21. COMPLETE ENTERPRISE PIPELINE

```
User Query
    ↓
Frontend (Streamlit/React)
    ↓
FastAPI Backend
    ↓
LangChain / LangGraph
    ↓
Retriever
    ↓
Hybrid Search
    ↓
```

```
Reranker
    ↓
Top Context
    ↓
Prompt Template
    ↓
LLM API
    ↓
Grounded Response
    ↓
Evaluation + Logging
```

---

22. WHAT YOU MUST MASTER FOR PIF

TOP PRIORITY SKILLS

Skill	Importance
RAG pipelines	CRITICAL
Chunking strategy	CRITICAL
Retrieval quality	CRITICAL
Metadata handling	VERY HIGH
Hybrid retrieval	VERY HIGH
LangChain orchestration	VERY HIGH
Agent workflows	VERY HIGH
PDF parsing	HIGH
Evaluation frameworks	HIGH
FastAPI	HIGH

---

23. THE MOST IMPORTANT MINDSET SHIFT

Beginner mindset:

```
"How do I call ChatGPT?"
```

Professional GenAI Engineer mindset:

```
"How do I architect a reliable retrieval and reasoning system?"
```

That difference is what separates:

- tutorial followers, from:
- enterprise AI engineers.